

Table of Content

No	Content	Page number
1	1.0 Introduction	
2	2.0 Assembly Language with detailed description	
3	3.0 Overview of a processor	
	3.0.1 Overview of x86 processor	
	3.1 Comparison between processors	
4.	4.0 Conclusion	
5.	5.0 References	
6.	6.0 Appendix	

List of Tables

List of Figures

List of Abbreviation

No	Abbreviation	Full Name
1	1.0 Introduction	
2	Assembly Language	
3	Overview of processors	
4	Comparison of processors	
5	Conclusion	

Part A

Assembly Language

```
.data
meseq: .asciiz"please choose a equation(1-6):\n"
eq1: .asciiz"1.(A OR B) AND C\n"
eq2: .asciiz"2.(A NAND B) NOR C\n"
eq3: .asciiz"3.(A XOR B) AND C\n"
eq4: .asciiz"4.(A + B)-C\n"
eq5: .asciiz"5.(A X B) + C\n"
eq6: .asciiz"6.(A - B) X C\n"
mes: .asciiz "please enter three integers\n"
value1: .asciiz"ENTER FIRST VALUE,A:\n"
value2: .asciiz"ENTER SECOND VALUE,B:\n"
value3: .asciiz"ENTER THIRD VALUE,C:\n"
result: .asciiz"RESULT:\n"
ans: .word 0 #answer
err:.asciiz"invalid input\n"
```

```
.text
```

```
main:
```

```
#print eq1
li $v0,4
la $a0,eq1
syscall
```

```
#print eq2
li $v0,4
la $a0,eq2
syscall
```

```
#print eq3
li $v0,4
la $a0,eq3
syscall
```

```
#print eq4
li $v0,4
la $a0,eq4
syscall
```

```
#print eq5
li $v0,4
la $a0,eq5
syscall
```

```
#print eq6
li $v0,4
la $a0,eq6
syscall
```

```
#print meseq
li $v0, 4 #print string
la $a0, meseq #print string
syscall #print it
```

```
#read value from user
li $v0,5#read integer
syscall #read it
move $t0,$v0 #keep it in $t0
slt $t8,$t0,1
sgt $t7,$t0,6
beq $t8,1,error
beq$t7,1,error
#print mes
li $v0,4
la $a0,mes
syscall
```

```
#print value 1
li $v0,4
la $a0,value1
syscall
```

```
#read value1
li $v0,5
syscall
move $t1,$v0
```

```
#print value2
li $v0,4
la $a0,value2
syscall
```

```
#read value2
```

```
li $v0,5
syscall
move $t2,$v0
```

```
#print value3
li $v0, 4
la $a0,value3
syscall
```

```
#read value3
li $v0, 5
syscall
move $t3,$v0
#choosing equation
beq $t0,1,equation1
beq $t0,2,equation2
beq $t0,3,equation3
beq $t0,4,equation4
beq $t0,5,equation5
beq $t0,6,equation6
equation1:
or $t4,$t1,$t2# A OR B
and $t5,$t4,$t3#(A OR B) AND C
#store the result back the memory
sw $t5,ans
j answer# jump to answer
```

```
equation2:
# Calculate (A AND B)
and $t4, $t1, $t2 # A AND B
# Calculate (A NAND B) = NOT (A AND B)
nor $t5, $t4, $t4 # NOT (A AND B)
# Calculate (A NAND B) NOR C
nor $t6, $t5, $t3 # (A NAND B) NOR C
#store the result back the memory
sw $t6,ans
j answer
```

```
equation3:
# Calculate (A XOR B)
xor $t4, $t1, $t2 # A XOR B
# Calculate (A XOR B) AND C
and $t5, $t4, $t3 # (A XOR B) AND C
# Store the result in memory
```

```
sw $t5, ans
j answer
```

equation4:

```
# Calculate (A + B)
add $t4, $t1, $t2 # A + B
# Calculate (A + B) - C
sub $t5, $t4, $t3 # (A + B) - C
# Store the ans in memory
sw $t5, ans
j answer
```

equation5:

```
# Calculate (A * B)
mul $t4, $t1, $t2 # A * B
# Calculate (A * B) + C
add $t5, $t4, $t3 # (A * B) + C
# Store the ans in memory
sw $t5, ans
j answer
```

equation6:

```
# Calculate (A - B)
sub $t4, $t1, $t2 # A-B
# Calculate (A - B) * C
mul $t5, $t4, $t3 # (A -B) * C
# Store the result in memory
sw $t5, ans
j answer
```

error:

```
#print error
li $v0,4
la $a0,err
syscall
# Exit program
li $v0, 10      # Load exit syscall code
syscall         # Exit
```

answer:

```
lw $t9,ans #load ans to $t9
#print result
```

```

li $v0,4
la $a0,result
syscall

#read result
li $v0,1
move $a0,$t9
syscall

# Exit program
li $v0, 10      # Load exit syscall code
syscall        # Exit

```

INSTRUCTION SET

Arithmetic and Logic Instructions: or, and, nor, xor, add, sub, mul

Conditional/Branching Instructions: beq, bne

Load/Store Instructions: lw, sw

Data Transfer Instructions: move

System Call Instructions: syscall

INSTRUCTION FORMAT

R-Format: Used for arithmetic and logic instructions.

Opcode (6 bits): Specifies the operation code.

Source Register 1 (5 bits): The first source operand.

Source Register 2 (5 bits): The second source operand.

Destination Register (5 bits): The result destination register.

Shift Amount (5 bits): For shift operations.

I-Format: Used for immediate arithmetic/logic, branching, and load/store instructions.

Opcode (6 bits): Specifies the operation code.

Source Register (5 bits): The source register.

Destination Register (5 bits): The destination register or base register for load/store.

Immediate (16 bits): Immediate value for the instruction.

J-Format: Used for jump instructions.

Opcode (6 bits): Specifies the operation code.

Address (26 bits): Target address for the jump.

REGISTER

\$t0, \$t1, \$t2, \$t3: Temporary registers to hold input values.

\$t4, \$t5: Temporary registers to hold intermediate calculation results.

\$t7, \$t8: Temporary registers used for branching validation.

\$t9: Temporary register to hold the final result.

\$v0: Register holding the syscall service number.

\$a0, \$a1: Argument registers for syscall.

\$v1: Register used for branching in error handling.

\$zero: Register that always holds the value 0.

SYSTEM CALL

li \$v0, 4: Load immediate value 4 into \$v0 to indicate a print string syscall.

li \$v0, 5: Load immediate value 5 into \$v0 to indicate a read integer syscall.

li \$v0, 10: Load immediate value 10 into \$v0 to indicate an exit syscall.

ADDRESSING MODES

Immediate Addressing: Used for loading immediate values into registers.

Base Addressing (Displacement): Used for accessing data in memory using a base register (load/store instructions).

Register Addressing: Used for arithmetic and logical operations involving registers.

PC-relative Addressing (Branching): Used for branching instructions where the target address is calculated relative to the current program counter.

Pseudocode

Data Section:

```
`meseq`: "Please select an equation by typing an integer from 1 to 6:"  
`eq1`: "1. Calculate (A OR B) AND C"  
`eq2`: "2. Calculate (A NAND B) NOR C"  
`eq3`: "3. Calculate (A XOR B) AND C"  
`eq4`: "4. Calculate (A + B) - C"  
`eq5`: "5. Calculate (A * B) + C"  
`eq6`: "6. Calculate (A - B) * C"  
`mes`: "Please enter three integers:"  
`value1`: "Enter the first integer (A):"  
`value2`: "Enter the second integer (B):"  
`value3`: "Enter the third integer (C):"  
`result`: "Result:"  
`ans`: Initialized to 0 and holds the answer of calculations  
`error`: "Invalid input."
```

Text Section:

```
`main`:  
Display equation options and instructions.  
Read the user's equation choice into `t0`.  
  
If the choice is less than 1 or greater than 6:  
Display an error message.  
Exit the program.  
  
Display instructions to enter three integers.  
Prompt user to input first integer and read the first number into `t1`.  
Prompt user to input second integer and read the second number into `t2`.  
Prompt user to input third integer and read the third number into `t3`.  
  
If the user chooses equation 1:  
Calculate (A OR B) AND C.  
Store the result in `t5`.  
Jump to the `answer` section.  
  
If the user chooses equation 2:  
Calculate NOT (A AND B) NOR C.  
Store the result in `t6`.  
Jump to the `answer` section.
```

If the user chooses equation 3:
Calculate $(A \text{ XOR } B) \text{ AND } C$.
Store the result in ``$t5``.
Jump to the ``answer`` section.

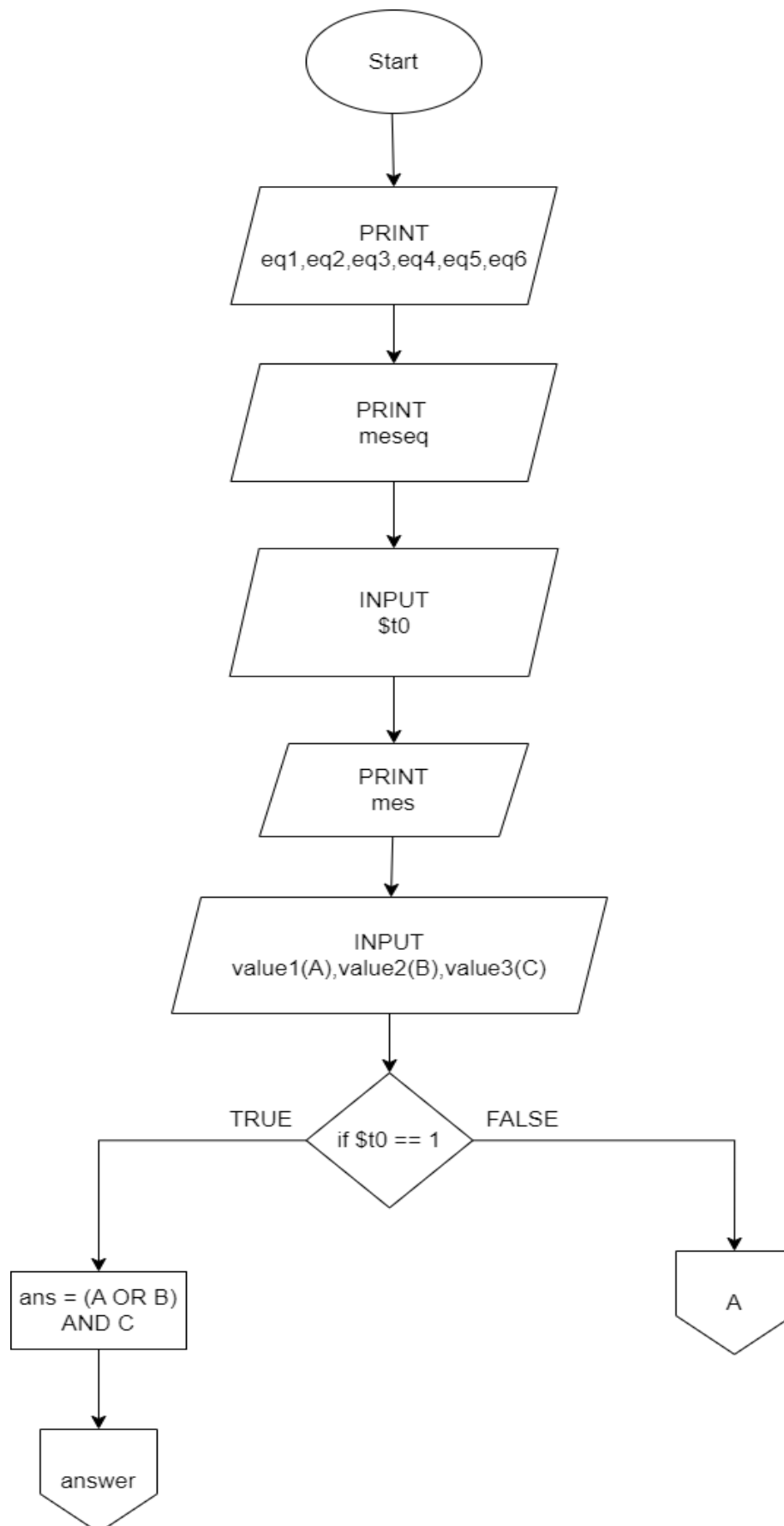
If the user chooses equation 4:
Calculate $(A + B) - C$.
Store the result in ``$t5``.
Jump to the ``answer`` section.

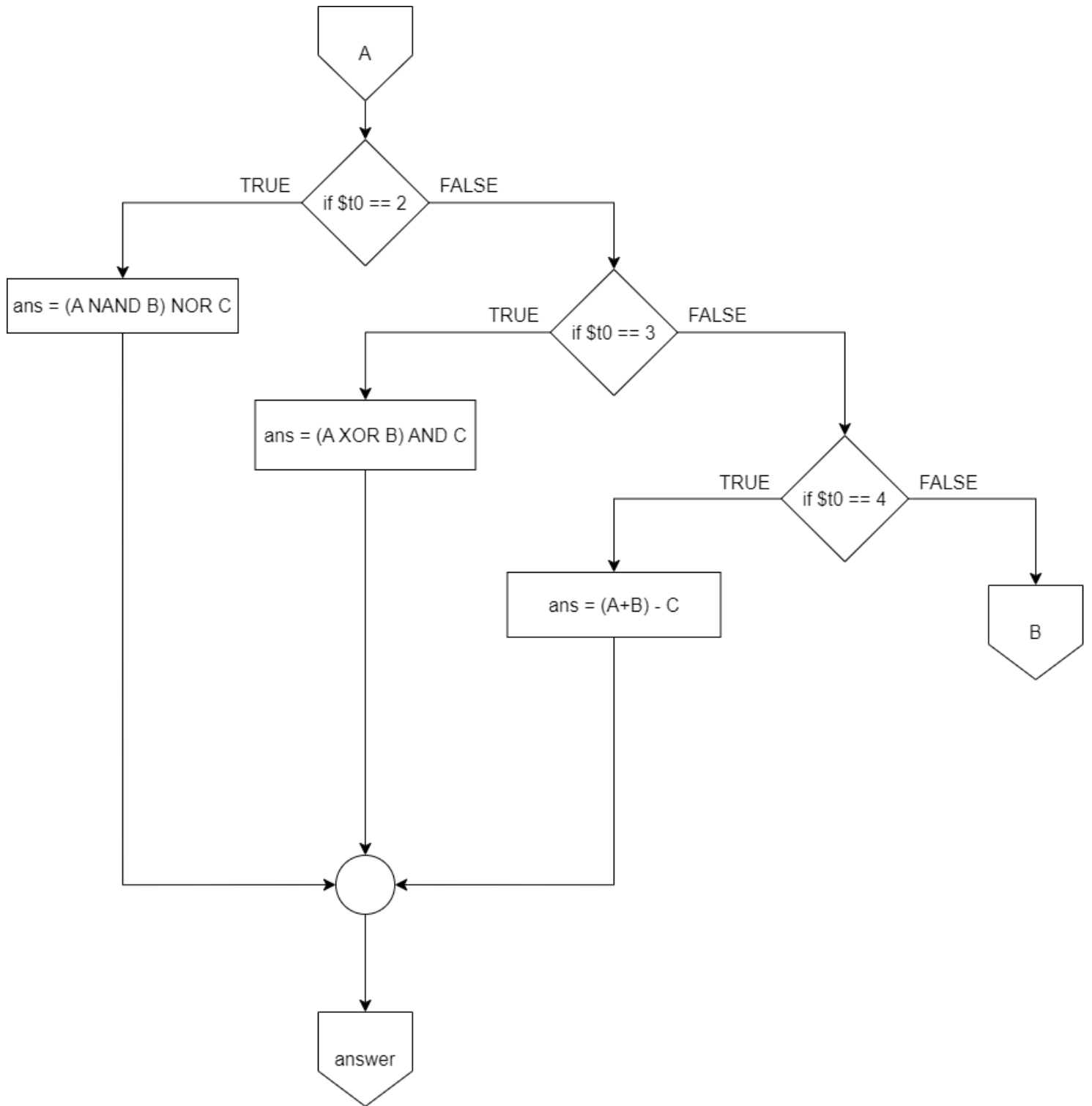
If the user chooses equation 5:
Calculate $(A * B) + C$.
Store the result in ``$t5``.
Jump to the ``answer`` section.

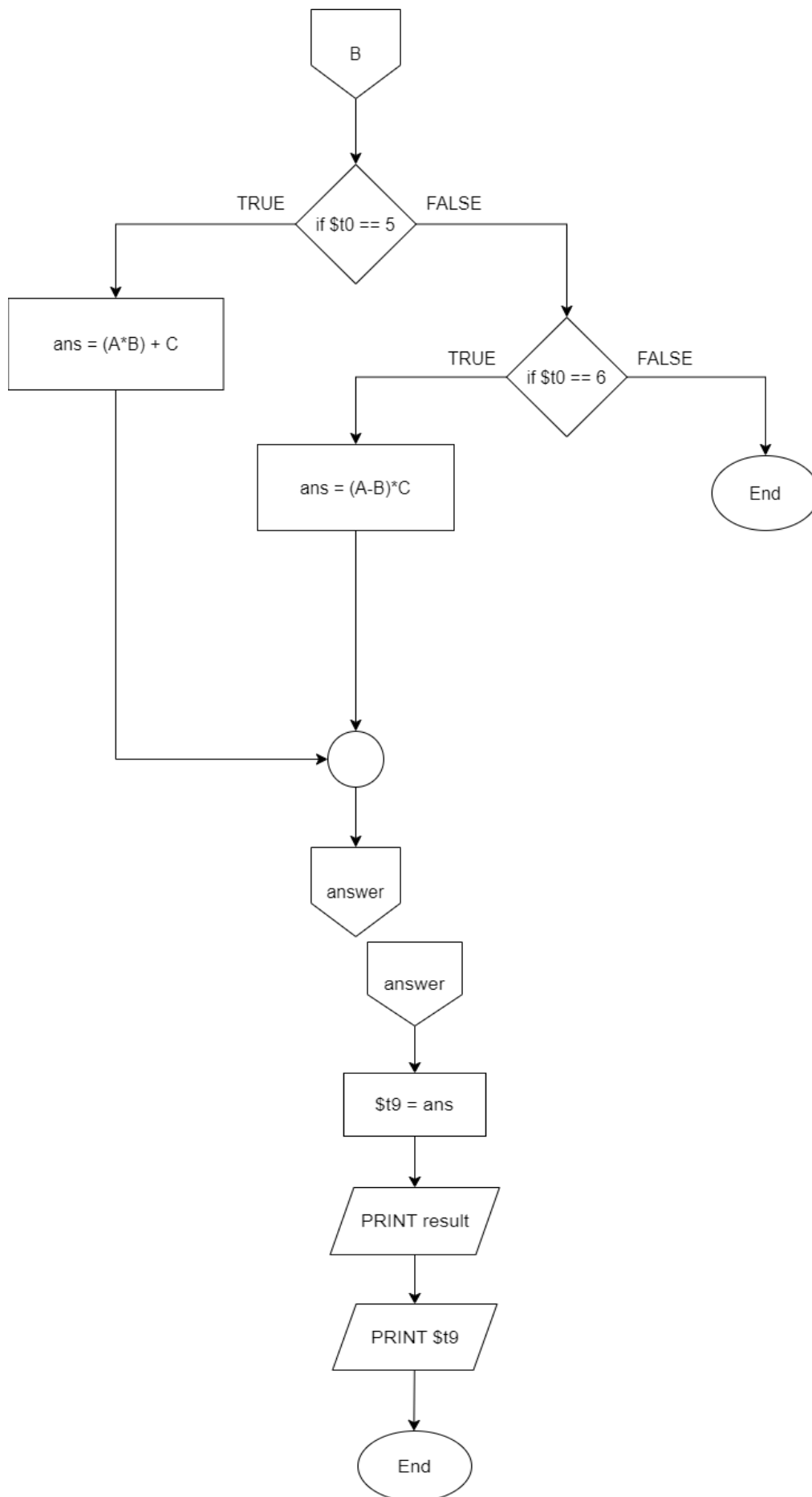
If the user chooses equation 6:
Calculate $(A - B) * C$.
Store the result in ``$t5``.
Jump to the ``answer`` section.

``answer`` section:
Load the result from ``$t5`` into ``$t9``.
Display the "Result:" message.
Display the calculated result stored in ``$t9``.
Exit the program.

Flowchart







3.0 Overview of a processor

3.0.1 Overview of x86

Evolution of x86

The x86 or the 80x86 or 8086 family, developed by Intel, is a family of CISC (x86, 2023)[[1](#)]. Initially, x86 began with 8-bit instruction set, but evolved into 16-bit instruction set and 32-bit instruction sets. Under the x86 architecture, Intel has released various processors such as 8086, 80286, 80386, 80486, Pentium, Core, and Xeon series. x86 can be used to run almost any type of computer, supercomputers, desktops, servers, and laptops. The term “x86” originated from the Intel 8086 chip which had 86 at the end (x86, 2015). [\[2\]](#)

Instruction set

Being part of CISC processors, this means x86 can perform multiple and complex tasks with fewer instructions compared to RISC processors. High-level programs or statement languages can be easily converted into assembly or machine language by the compiler. However, the complexity of the instruction set can lead to slower performance and execution time compared to RISC. RISC, on the other hand, perform better with its simpler and limited number of instructions set. Nonetheless, x86 is still used to this day. (Difference Between RISC and CISC, n.d.) [\[3\]](#)

Registers

The processor registers are the primary tools for writing programmes in x86 assembly. Utilising registers instead of memory is quicker and more efficient than using memory. The number of registers varies according to the processor generation. Nonetheless, the x86 architecture has 8 GPR which Address calculations, arithmetic and logical calculations are usually performed with these registers (Azu, 2019). [\[4\]](#) The segment address of various items is stored in the 6 available segment registers, 1 flag register and an instruction pointer(x86 Assembly/X86 architecture, 2023). [\[5\]](#)

Register	Description	Register	Description
eax	Accumulator	esi	Source index register
ebx	Base register	edi	Destination index register
ecx	Counter register	edp	Base pointer register
edx	Data – register	esp	Stack pointer

Note: Data from “x86 Architecture”, by Microsoft, 2022

Endianness

Little-endian byte ordering is used by the x86 processors. This means that the lowest address of the integer stores LSB and the MSB is stored at the highest address.

Byte 3	Byte 2	Byte 1	Byte 0
MSB			LSB

Extension

x86 architecture is later extended into x64. This extension provides two mode of operation, the 64-bit mode and compatibility mode, allowing users to run both 16-bit and 32-bit applications. The older executables run with practically no performance costs as the whole x86 instruction set is still implemented in the x64 one. Furthermore, x64 expands the number of GPR to 16, allowing greater functionality. The entire amount of memory accessible to x64 is 2^{64} bytes, or 16 billion gigabytes of memory. This is useful for powering supercomputers and other machines that need access to extensive resources (Marijan, 2022). [6]

Legacy and Compatibility

- **Backward compatibility**

A hardware or software system can successfully utilise interfaces and data from earlier versions of the system or with hardware is it is backward compatible. This can be more easily achieved if the previous versions are designed to be forward compatible or extensible that allows the addition of new features. This can be seen in the x86 architecture. This means that x86 processors can execute the same binary software instructions as their forerunners, without the need for new programs or OS (Hanna, n.d.).

Protected Mode

Protected Mode is the primary mode of operation for Intel-based x86 microprocessors. This mode prevents the address space of any application from writing over another application's address space. This restriction allows different applications written by different developers to execute on the same computer. In the absence of such computer, a malicious program can write over the space of another program and cause malfunction or crashing. A selector in a segment register is used to pick a descriptor from the descriptor table. The descriptor contains details about the segment, such as its starting address, length, and access privileges. Protected mode makes use of this data to regulate how programs access memory (Brans, n.d.). [8]

Applications

- **Personal Computers**

Majority of PCs in use today still use x86 operating systems and CPUs. The compatibility of x86 with the abundance software library and the vast hardware components make x86-based PCs accessible to a broad user base. For instance, x86 is used in Microsoft Win32 in 32-bit flat mode.

- **Servers**

x86 processors are commonly used in servers. Because x86 processors are designed for general-purpose computing, a variety of tasks such as web hosting, database management, and file sharing can be accomplished. Thus, if a general-purpose server is used, x86 will be the best choice. However, RISC processor may be a better choice if a high-performance server is involved (The Ultimate Guide for Server Processors (2022), 2022). [9]

Comparison between x86 and MIPS

Complexity

- x86

The x86 architecture supports a wide range of instructions, which can be quite complex, allowing for more powerful instructions but often results in higher power consumption. Due to complexity of instructions, x86 processors feature deeper pipelines, which can introduce challenges related to branch prediction and hazards (Hennessy, 2017).

- MIPS

The MIPS architecture emphasizes a smaller, more streamlined set of simple and efficient instructions. In contrast of x86, MIPS has simpler instructions which cause it to have shorter pipelines. MIPS also have a higher execution speed and lower power consumption.

Endianness

- x86

Most x86 processors use the little-endian byte order, where the least significant byte (LSB) is stored at the lowest memory address of a multi-byte data type. Little-endian is often considered more efficient for multi-byte memory accesses, especially when dealing with integers and floating-point numbers.

- MIPS

MIPS supports both little-endian and big-endian modes, allowing higher flexibility based on the user's requirements. In big-endian mode, the least significant byte (LSB) is stored at the highest memory address. The choice of endianness might be influenced by factors such as hardware design, compatibility with existing software, and the intended use of the device.

Register Usage

- x86

x86 architecture has a relatively small number of general-purpose registers. However, it compensates for this limitation with complex addressing modes and memory access instructions for example general-purpose registers, special-purpose registers, floating-point registers, and Single Instruction, Multiple Data (SIMD) registers.

- **MIPS**

MIPS32 architecture however doesn't have SIMD registers. But MIPS still have general-purpose registers, special-purpose registers, and floating-point registers. This promotes register-based computation and reduces the need for memory access, which can enhance performance.

Application

- **x86**

The x86 architecture is widely used in personal computers, laptops, servers, and workstations. Its compatibility with the Windows and Linux operating systems and its history make it a dominant architecture in the general-purpose computing space.

- **MIPS**

MIPS32 architecture has been historically popular in embedded systems, networking equipment, and certain specialized applications like routers and digital signal processors (DSPs).

x86	MIPS32
More complexed instructions	Simpler instructions
Little-endian	Little-endian and Big-endian
Have SIMD registers	No SIMD registers
Widely used in personal devices/workstations	Commonly used in various embedded and specialized applications

JL Hennessy, DA Patterson (2011) *Computer architecture: a quantitative approach*

[https://scholar.google.com.my/scholar?q=Hennessy,+J.+L.,+%26+Patterson,+D.+A.+\(2017\).+Computer+Architecture:+A+Quantitative+Approach+\(6th+ed.\).+Morgan+Kaufmann.&hl=en&as_sdt=0&as_vis=1&oi=scholart](https://scholar.google.com.my/scholar?q=Hennessy,+J.+L.,+%26+Patterson,+D.+A.+(2017).+Computer+Architecture:+A+Quantitative+Approach+(6th+ed.).+Morgan+Kaufmann.&hl=en&as_sdt=0&as_vis=1&oi=scholart)

Conclusion

Appendix